

PROMPT 02: CI + SEGURIDAD — Workflows, Protección y Validación

****Contexto verificable (2026-10-02)****

- Repo: `RigohI/auto-healing-agent`, branch `main`, HEAD `f7522c8`
- Stack: Rust + workers-rs 0.8 + GitHub Actions
- SoT: `docs/GOVERNANCE.md` (MIN_CONFIDENCE=0.55, MAX_RISK=0.45)
- Restricción: ****Ningún LLM****, ningún código generado por la red.

ALCANCE (Fases 5-7)

FASE 5 — CI Real

****Problema:**** Los workflows actuales (`security.yml`, `repair-validation.yml`, `auto-repair.yml`) son `echo`.

****Workflows requeridos:****

1. `ci.yml` (Build + Test + Lint)

```
name: CI on: push: branches: [main] paths-ignore: ['docs/**', 'legacy/**', '**.md'] pull_request: branches: [main] jobs: workspace: runs-on: ubuntu-latest steps: - uses: actions/checkout@v4 - uses: actions-rust-lang/setup-rust-toolchain@v1 with: toolchain: stable components: rustfmt, clippy - uses: Swatinem/rust-cache@v2 - name: cargo fmt --check run: cargo fmt --all -- --check - name: cargo check --workspace run: cargo check --workspace --all-targets - name: cargo test --workspace run: cargo test --workspace --all-targets - name: cargo clippy --workspace run: cargo clippy --workspace --all-targets -- -D warnings - name: cargo audit run: cargo audit worker: runs-on: ubuntu-latest steps: - uses: actions/checkout@v4 - uses: actions-rust-lang/setup-rust-toolchain@v1 with: toolchain: stable target: wasm32-unknown-unknown - uses: Swatinem/rust-cache@v2 - name: cargo check worker run: | cd worker cargo check --manifest-path Cargo.toml --target wasm32-unknown-unknown --release - name: Build WASM run: | cd worker cargo build --release --target wasm32-unknown-unknown
```

2. `security.yml` (Auditoría + Secretos)

```
name: Security on: push: branches: [main] pull_request: branches: [main] schedule: - cron: '0 0 * * *' # Diario a medianoche jobs: audit: runs-on: ubuntu-latest steps: - uses: actions/checkout@v4 - uses: actions-rust-lang/setup-rust-toolchain@v1 - uses: Swatinem/rust-cache@v2 - run: cargo audit --deny warnings secret-scan: runs-on: ubuntu-latest steps: - uses: actions/checkout@v4 - uses: trufflesecurity/trufflehog@main with: path: . base: main head: HEAD filter: files wrangler-check: runs-on: ubuntu-latest steps: - uses: actions/checkout@v4 - name: Check wrangler.toml run: | # Verificar que no hay REPLACE_WITH_* if grep -r "REPLACE_WITH_" worker/wrangler.toml; then echo "::error::wrangler.toml contains REPLACE_WITH_ placeholders" exit 1 fi # Verificar que KV namespace ID es real (formato UUID) if ! grep -E "[a-f0-9]{32}$" worker/wrangler.toml; then echo "::error::KV namespace ID is not a valid UUID" exit 1 fi
```

3. `repair-validation.yml` (Validación de Reparaciones)

```
name: Repair Validation on: workflow_run: workflows: ["CI"] types: [completed] branches: [main] jobs: validate: if: ${github.event.workflow_run.conclusion == 'success'} runs-on: ubuntu-latest permissions: contents: read pull-requests: read steps: - uses: actions/checkout@v4 - uses: actions-rust-lang/setup-rust-toolchain@v1 - uses: Swatinem/rust-cache@v2 - name: Run repair tests run: | cargo test --package
```

```

repair_operators -- --nocapture cargo test --package repair_nn_core -- --nocapture -
name: Verify WEIGHT_COUNT run: | # Extraer WEIGHT_COUNT del código y docs
CODE_COUNT=$(grep "pub const WEIGHT_COUNT" crates/repair_nn_core/src/lib.rs | grep -oE
'[0-9]+') DOCS_COUNT=$(grep -E "WEIGHT_COUNT.*[0-9]+" docs/ARCHITECTURE.md | grep -oE
'[0-9]+' | head -1) if [ "$CODE_COUNT" != "$DOCS_COUNT" ]; then echo
"::error::WEIGHT_COUNT mismatch: code=$CODE_COUNT, docs=$DOCS_COUNT" exit 1 fi

```

4. `auto-repair.yml` (Orquestación - OPCIONAL)

****Nota:**** Este workflow ****NO** debe ser autoridad de VERIFY** (solo GitHub Actions).

```

name: Auto-Repair on: workflow_dispatch: inputs: incident_id: description: 'Incident
ID' required: true repository: description: 'Repository' required: true dry_run:
description: 'Dry run (no apply)' required: false default: 'true' jobs: repair: if:
github.event_name == 'workflow_dispatch' runs-on: ubuntu-latest permissions: contents:
read steps: - uses: actions/checkout@v4 - uses:
actions-rust-lang/setup-rust-toolchain@v1 - uses: Swatinem/rust-cache@v2 - name: Dry
run if: inputs.dry_run == 'true' run: | cargo run --bin repair_cli -- --dry-run
--incident ${{ inputs.incident_id }} --repo ${{ inputs.repository }} - name: Apply
repair if: inputs.dry_run == 'false' run: | # REQUERIMIENTO: Este paso NO debe declarar
PASS # Solo propone reparación, GitHub Actions valida cargo run --bin repair_cli --
--apply --incident ${{ inputs.incident_id }} --repo ${{ inputs.repository }} echo
"::notice::Repair proposed. GitHub Actions is the VERIFY authority."

```

****Permisos:****

```

# En TODOS los workflows: permissions: contents: read # Solo lectura para seguridad #
NO usar contents: write en workflows automáticos

```

****Concurrency:****

```

concurrency: group: ${{ github.workflow }}-${{ github.ref }} cancel-in-progress: true

```

FASE 6 — Protección de Rama

****Problema:**** `gh api repos/Rigohl/auto-healing-agent/branches/main/protection` devuelve 403.

****Acciones:****

1. Verificar token actual:

```

gh auth status gh api repos/Rigohl/auto-healing-agent/rulesets

```

2. Si 403 (token sin admin):

```

# Configuración lista para aplicar (requiere token con repo:admin) cat >
/tmp/branch_protection.json << 'EOF' { "required_status_checks": { "strict": true,
"contexts": ["CI", "Security", "Repair Validation"] }, "enforce_admins": true,
"required_pull_request_reviews": { "dismiss_stale_reviews": true,
"require_code_owner_reviews": true, "required_approving_review_count": 1 },
"restrictions": null, "required_linear_history": true, "allow_force_pushes": false,
"allow_deletions": false } EOF # Aplicar cuando se tenga token: gh api
repos/Rigohl/auto-healing-agent/branches/main/protection \ --method PUT \ --input
/tmp/branch_protection.json

```

3. Si no se puede verificar:

```

Estado: UNVERIFIABLE (token sin alcance admin) Acción: Dejar configuración lista en
docs/BRANCH_POLICY.md

```

****Reglas de branch:****

- `main` protegida
- Requiere:
- CI ■
- Security ■
- Repair Validation ■
- 1 review de code owner
- Historia lineal
- No force pushes

FASE 7 — Limpieza

****Elementos a eliminar:****

Elemento	Razón	Evidencia	Estado
`repair_nn_wasm`	66 líneas, 0 tests, 0 consumidores. Worker enlaza `repair_nn_core` directamente.		
`crates/repair_nn_wasm/`			Pendiente
Operadores Node/TS	Reemplazados por allowlist Rust (FASE 1)		
`repair_operators/src/lib.rs`			Pendiente
`feature_engine:37` (linear)	Sin integrador, slot inerte		
`feature_engine/src/lib.rs`			Pendiente
`feature_engine:14` (vercel)	Sin integrador, slot inerte		
`feature_engine/src/lib.rs`			Pendiente
`set-github-secrets.sh` (líneas HF/VERCEL)	Servicios fuera del camino Rust		
`scripts/set-github-secrets.sh`			Pendiente
`legacy/CONFIG.md:9` (VERCEL_ORG_ID)	Token real en código		
`legacy/CONFIG.md`			Pendiente

****Acciones:****

```
# 1. Eliminar crate repair_nn_wasm rm -rf crates/repair_nn_wasm # Actualizar Cargo.toml
sed -i '/repair_nn_wasm/d' Cargo.toml # 2. Eliminar operadores Node/TS de
repair_operators # Mantener solo operadores Rust (FASE 1) # 3. Eliminar slots inertes
de feature_engine # Documentar en DISCREPANCIES.md # 4. Limpiar
scripts/set-github-secrets.sh sed -i '/EXA_API_KEY\|LINEAR_API_KEY\|VERCEL_\|HF\/d'
scripts/set-github-secrets.sh # 5. Rotar VERCEL_ORG_ID en legacy/CONFIG.md sed -i
's/VERCEL_ORG_ID/[REDACTED]/g' legacy/CONFIG.md
```

****Archivos a AÑADIR:****

```
# CODEOWNERS echo "@Rigohl *" > .github/CODEOWNERS # dependabot.yml cat >
.github/dependabot.yml << 'EOF' version: 2 updates: - package-ecosystem: "cargo"
directory: "/" schedule: interval: "weekly" open-pull-requests-limit: 10 reviewers: -
"Rigohl" labels: - "dependencies" - "rust" EOF # SECURITY.md cat > SECURITY.md << 'EOF'
# Security Policy ## Reporting Vulnerabilities Please report security vulnerabilities
via GitHub Security Advisories. ## Supported Versions - Only the latest version of
`main` is supported. ## Security Checks - `cargo audit` runs on every push/PR -
TruffleHog scans for secrets daily - No LLM code generation (see NO_LLM_POLICY.md) EOF
```

PROHIBICIONES ABSOLUTAS

- ■ No declarar `PASS`, `READY` ni `deployed` sin evidencia ejecutada
- ■ No usar `contents: write` en workflows automáticos
- ■ No crear workflows que declaren autoridad de VERIFY
- ■ No usar secrets en logs o outputs
- ■ No eliminar `.github/CODEOWNERS` si existe

SALIDA REQUERIDA

1. Workflows completos (`ci.yml`, `security.yml`, `repair-validation.yml`, `auto-repair.yml`)
2. Permisos configurados (`contents: read` en todos)
3. Concurrency configurado
4. Estado de protección de rama: `CONFIGURADO` o `UNVERIFIABLE`
5. Tabla de eliminaciones con justificación
6. Archivos añadidos: `CODEOWNERS`, `dependabot.yml`, `SECURITY.md`
7. Cada comando ejecutado con salida real o `NOT RUN`

****Estado final:**** `READY` o `BLOCKED` con lista exacta de pendientes.